Dictionary Case Studies for Data Science

Python Session 4: Dictionaries | Data Science Practice | 4 case studies

Classroom exercise. Complete these case studies in VS Code during the session. Your trainer will tell you which ones to attempt and will check your output on your screen.

How to work

Step	What to do
1	Create a project folder in VS Code and activate your **virtual environment**. The terminal must show the environment name before you run anything.
2	Create **one file per case study**, using the file name given at the top of each case study.
3	Do the tasks **in order**. Run your program after every task and compare it with the expected output.
4	Your output must match the expected output **exactly**, including spaces and alignment.
5	Call your trainer when you finish, and try the challenge if time is left.

The case studies

#	Case study	File name
1	Dataset Profiler	`dataset_profiler.py`
2	Data Cleaning and Encoding	`data_cleaning.py`
3	Customer Review Analysis	`review_analysis.py`
4	Sales GroupBy Report	`sales_report.py`

Remember: a dictionary holds `key: value` pairs. Use `d[key]` when the key must exist and `d.get(key, default)` when it may not. Loop with `for k, v in d.items()`.

Case Study 1: Dataset Profiler

File: `dataset_profiler.py` | Practises: dictionary of lists (columns), None for missing values, counting, summary statistics

A data scientist's first step with any new dataset is to **profile** it: how big is it, what is missing, and what do the numbers look like? Here a dataset of 8 students is stored the way pandas stores a table: a **dictionary where each key is a column name and each value is the list of that column's values**.

Data science link. This is what `df.shape`, `df.isna().sum()`, `df.describe()` and `df['city'].value_counts()` do in pandas. Today you build them by hand so you understand what happens underneath.

1. The data

Student dataset (a blank cell below means the value is missing and must be stored as `None`)

student_id	age	hours	city	score
101	21	5.5	Kochi	78
102	22	3.0	Kollam	65
103	**missing**	7.0	Kochi	88
104	20	2.5	Thrissur	52
105	23	6.0	Kollam	74
106	21	4.5	Kochi	70
107	22	**missing**	Kollam	69
108	24	8.0	Kochi	91

2. Tasks

Run the program after every task.

Task	What your program must do
1. Create the dataset	Store the dataset (Section 2) in a dictionary named `data`. Each key is a column name and each value is a list. Missing values are written as `None`.
2. Shape of the data	Print the title and a line of 40 equals signs. Then print the number of rows (the length of any column), the number of columns (the length of the dictionary) and the column names joined by a comma and a space.
3. Missing values per column	Print a blank line and the heading `MISSING VALUES`. Loop over the columns and count how many values are `None` in each. Print the column name (width 12) and the count (width 3). Test with `is None`.
4. Numeric summary	For the numeric columns `age`, `hours` and `score` only, ignore the `None` values and print the count, mean, minimum and maximum. Use the widths in the expected output (column name 8, then 6, 8, 7, 7) and two decimals for the mean.
5. Category frequency	For the `city` column build a dictionary of city and number of students using the counting pattern. Print the heading `CITY FREQUENCY` and one line per city in the order the cities first appear: the city name (width 10), the count (width 2), two spaces and a bar of `#` characters.

3. Expected output

Your output must match this exactly.

```
DATASET PROFILE
=======================================
Rows: 8
Columns: 5
Column names: student_id, age, hours, city, score

MISSING VALUES
student_id    0
age           1
hours         1
city          0
score         0

NUMERIC SUMMARY
Column   Count    Mean    Min    Max
age          7   21.86     20     24
hours        7    5.21    2.5    8.0
score        8   73.38     52     91

CITY FREQUENCY
Kochi      4  ####
Kollam     3  ###
Thrissur   1  #
```

Challenge for fast finishers. Convert the column-oriented `data` into a **list of row dictionaries**, one dictionary per student (the shape pandas calls `to_dict('records')`). Use a loop over `range(rows)` and an inner loop over the columns. Print the first two rows.

Expected challenge output (printed after the output above):

```
FIRST TWO ROWS
{'student_id': 101, 'age': 21, 'hours': 5.5, 'city': 'Kochi', 'score': 78}
{'student_id': 102, 'age': 22, 'hours': 3.0, 'city': 'Kollam', 'score': 65}
```

Case Study 2: Data Cleaning and Encoding

File: `data_cleaning.py` | Practises: list of dictionaries (rows), missing-value imputation, label encoding with a dictionary, grouping

A telecom company wants to understand which customers **churn** (leave the service). Before any machine learning model can use the data, a data scientist must **fill the gaps** and **convert text categories into numbers**. Each customer is a dictionary in a list.

Data science link. This is `fillna(mean)`, `map()` / `LabelEncoder` and `groupby()` in pandas. Machine learning models need numbers without gaps, so cleaning and encoding come before any model.

1. The data

Customer records (a missing usage value is stored as `None`)

id	plan	usage (GB)	churned
1	Basic	12.5	No
2	Premium	14.58	Yes
3	Standard	8.0	No
4	Basic	15.0	Yes
5	Premium	22.5	No
6	Standard	14.58	No
7	Basic	9.5	Yes
8	Premium	20.0	No

Codes to use: plan: Basic 0, Standard 1, Premium 2. Churned: No 0, Yes 1.

2. Tasks

Run the program after every task.

Task	What your program must do
1. Create the records	Store the eight customers (Section 2) in a list named `records`. Each customer is a dictionary with the keys `id`, `plan`, `usage` (monthly data in GB, or `None` if missing) and `churned`.
2. Check the data	Loop over the records and count the customers whose `usage` is missing, while adding up the valid usage values. Print the heading `CHECK THE DATA`, the number of missing values, and the mean of the valid values rounded to 2 decimals using `round()`.
3. Fill the gaps (imputation)	Replace every missing `usage` with the mean you calculated, directly inside the dictionaries. Then print a blank line, the heading `CLEANED DATA` and a table: id (width 4), plan (width 10), usage (width 7, two decimals) and churned (width 9), with a header line in the same widths.
4. Encode the categories	Models need numbers. Create two dictionaries: `plan_codes` (Basic 0, Standard 1, Premium 2) and `churn_codes` (No 0, Yes 1). Add two new keys to every customer: `plan_code` and `churn_flag`. Print the heading `ENCODED DATA` and a table with id (width 4), plan_code (width 10), usage (width 8, two decimals) and churn_flag (width 12), with a header line.
5. Churn rate by plan	Find the overall churn rate (churned customers divided by all customers, as a percentage to 1 decimal). Then group by plan: for each plan keep a list of two numbers, the customers on that plan and the customers who churned. Print the overall rate, the heading `CHURN RATE BY PLAN` and one line per plan in alphabetical order: plan (width 10), `churned/total`, two spaces and the percentage (width 5, one decimal) followed by `%`.

3. Expected output

Your output must match this exactly.

```
CHECK THE DATA
Missing usage values: 2
Mean usage (valid values): 14.58

CLEANED DATA
ID  Plan        Usage  Churned
1   Basic       12.50       No
2   Premium     14.58      Yes
3   Standard     8.00       No
4   Basic       15.00      Yes
5   Premium     22.50       No
6   Standard    14.58       No
7   Basic        9.50      Yes
8   Premium     20.00       No

ENCODED DATA
ID   plan_code   usage  churn_flag
1            0   12.50           0
2            2   14.58           1
3            1    8.00           0
4            0   15.00           1
5            2   22.50           0
6            1   14.58           0
7            0    9.50           1
8            2   20.00           0

Overall churn rate: 37.5%
CHURN RATE BY PLAN
Basic     2/3   66.7%
Premium   1/3   33.3%
Standard  0/2    0.0%
```

Challenge for fast finishers. Writing the plan codes by hand does not scale to 50 categories. Write a function `build_encoder(values)` that takes a list of category values and **returns** a dictionary giving each unique value a code (0, 1, 2 ...) in alphabetical order. Collect the plans from the records into a list, call your function and print the result.

Expected challenge output (printed after the output above):

```
AUTO ENCODER
{'Basic': 0, 'Premium': 1, 'Standard': 2}
```

Case Study 3: Customer Review Analysis

File: `review_analysis.py` | Practises: word frequency, stop words, a sentiment lexicon dictionary, sorting (count, word) tuples, labels

An online shop has collected six customer reviews. The manager wants to know **which words customers use most** and **whether each review is positive or negative**. A dictionary of word scores (a **sentiment lexicon**) lets you score text without any machine learning model.

Data science link. This is the start of **Natural Language Processing**: word counts like `collections.Counter` or `CountVectorizer`, and a lexicon-based sentiment score. The same ideas power review analysis at large companies.

1. The data

Reviews

#	Review text
1	The delivery was fast and the product is great
2	Terrible service and the product arrived broken
3	Good quality but the delivery was late
4	I love this product and the support team is excellent
5	Poor packaging and slow delivery
6	Great product and fast delivery very happy

Stop words (skip these): the, and, is, was, but, i, this, very, a

Sentiment lexicon (word: score)

Positive words	Score	Negative words	Score
good	1	slow	-1
great	2	late	-1
excellent	3	poor	-2
love	2	bad	-2
fast	1	broken	-2
happy	2	terrible	-3

2. Tasks

Run the program after every task.

Task	What your program must do
1. Create the data	Store the six reviews in a list named `reviews`, the stop words in a list named `stop_words`, and the sentiment lexicon in a dictionary named `lexicon` (all in Section 2).
2. Top 5 words	Count how many times each word appears in all the reviews. Convert each review to lowercase with `.lower()`, split it into words with `.split()`, and skip the stop words. Build a list of `(count, word)` tuples, sort it with `sort(reverse=True)`, and print the heading `TOP 5 WORDS`, a line of 20 dashes and the top 5 as word (width 12) and count (width 3).
3. Sentiment score of each review	For every review add up the lexicon score of its words (a word that is not in the lexicon scores 0, so use `get(word, 0)`). A total above 0 is **Positive**, below 0 is **Negative**, and 0 is **Neutral**. Keep the scores in a list `scores` and the labels in a list `labels`. Print a blank line, the heading `SENTIMENT PER REVIEW`, 20 dashes and one line per review in the form `Review 1: 3 Positive` (score in a width of 3).
4. Summary	Count how many reviews fall into each label using a dictionary. Print a blank line, the heading `SUMMARY`, then each label in alphabetical order (label width 10, then the count) and finally the average score to two decimals.

3. Expected output

Your output must match this exactly.

```
TOP 5 WORDS
--------------------
product       4
delivery      4
great         2
fast          2
terrible      1

SENTIMENT PER REVIEW
--------------------
Review 1:   3  Positive
Review 2:  -5  Negative
Review 3:   0  Neutral
Review 4:   5  Positive
Review 5:  -3  Negative
Review 6:   5  Positive

SUMMARY
Negative  2
Neutral   1
Positive  3
Average score: 0.83
```

Challenge for fast finishers. Which words appear in **negative reviews only**? Using the reviews, labels and stop words, collect the words that appear in Negative reviews and the words that appear in Positive reviews (skip stop words). Print the heading `WORDS ONLY IN NEGATIVE REVIEWS` and then those words, sorted and joined by a comma and a space.

Expected challenge output (printed after the output above):

```
WORDS ONLY IN NEGATIVE REVIEWS
arrived, broken, packaging, poor, service, slow, terrible
```

Case Study 4: Sales GroupBy Report

File: `sales_report.py` | Practises: grouping and aggregating with dictionaries, averages, finding the best, nested dictionary (pivot), tuple sorting

A company sells laptops and phones in four regions. The sales team lead wants a quick report: **how much did each region sell, what is the average sale, which region is best**, and a region-by-product table. Each sale is one dictionary in a list.

Data science link. This is `df.groupby('region')['amount'].sum()`, `.mean()` and `pivot_table()` in pandas. Group-by is the most used operation in data analysis.

1. The data

Sales (amount in Rs)

#	Region	Product	Amount
1	North	Laptop	55,000
2	South	Phone	18,000
3	North	Phone	22,000
4	East	Laptop	48,000
5	South	Laptop	52,000
6	West	Phone	15,000
7	North	Laptop	60,000
8	East	Phone	20,000
9	South	Phone	25,000
10	West	Laptop	45,000

2. Tasks

Run the program after every task.

Task	What your program must do
1. Create the sales data	Store the ten sales (Section 2) in a list named `sales`. Each sale is a dictionary with the keys `region`, `product` and `amount`.
2. Total sales by region	Group the sales by region: build a dictionary with the region as the key and the total amount as the value. Print the heading `TOTAL SALES BY REGION`, a line of 22 dashes and each region in alphabetical order with the name (width 8) and the total (width 10, with thousands separators).
3. Average sale by region	Build a second dictionary `counts` with the number of sales in each region. Print a blank line, the heading `AVERAGE SALE BY REGION`, 22 dashes and each region in alphabetical order with the name (width 8) and the average (width 12, thousands separators, two decimals).
4. Best region	Using a loop over `totals`, find the region with the highest total and print it with its total (thousands separators).
5. Region by product table	Build a **nested dictionary** `pivot`: region, then product, then the total amount. Print the heading `REGION x PRODUCT`, a header line (`Region` in width 8, `Laptop` and `Phone` right-aligned in width 10) and one line per region in alphabetical order. Use `get(product, 0)` in case a region has no sale of a product.

3. Expected output

Your output must match this exactly.

```
TOTAL SALES BY REGION
----------------------
East         68,000
North       137,000
South        95,000
West         60,000

AVERAGE SALE BY REGION
----------------------
East        34,000.00
North       45,666.67
South       31,666.67
West        30,000.00

Best region: North (137,000)

REGION x PRODUCT
Region       Laptop     Phone
East         48,000    20,000
North       115,000    22,000
South        52,000    43,000
West         45,000    15,000
```

Challenge for fast finishers. Rank the regions by total sales, highest first, and print each region's **share of the grand total** as a percentage with one decimal. Hint: build a list of `(total, region)` tuples and sort it with `sort(reverse=True)`. Print the heading `REGION SHARE OF SALES`, then the region (width 8) and the share (width 6) followed by `%`.

Expected challenge output (printed after the output above):

```
REGION SHARE OF SALES
North     38.1%
South     26.4%
East      18.9%
West      16.7%
```